

• Software Engineering

- Programme / Instructions
- +
- Documents

- Design / create / build
- Solution to Problems
- systematic & disciplined
- Development, test & maintain

Example: Making of your house.

- User Requirements
- Time & budget
- Material & labour
- Steps of construction
- Blueprint designing
- Changes / updates

• Characteristics of 's/w'

- ① It is engineered NOT manufactured
- ② Does not wear out
It deteriorates / downgrade / degrade due to heavy changes.
- ③ It is custom built.
- ④ Can't be touched (untouched)

◦ Software Myths → false beliefs

→ confusion

→ Incorrect information

① Management Myth

② Customer Myth

③ Practitioner Myth

① Management Myths

① we have all standards & procedures available for software development.

② Adding of the latest programs will improve s/w development.

③ Adding up of more programmers to s/w development can help meet deadlines (In case of lag).

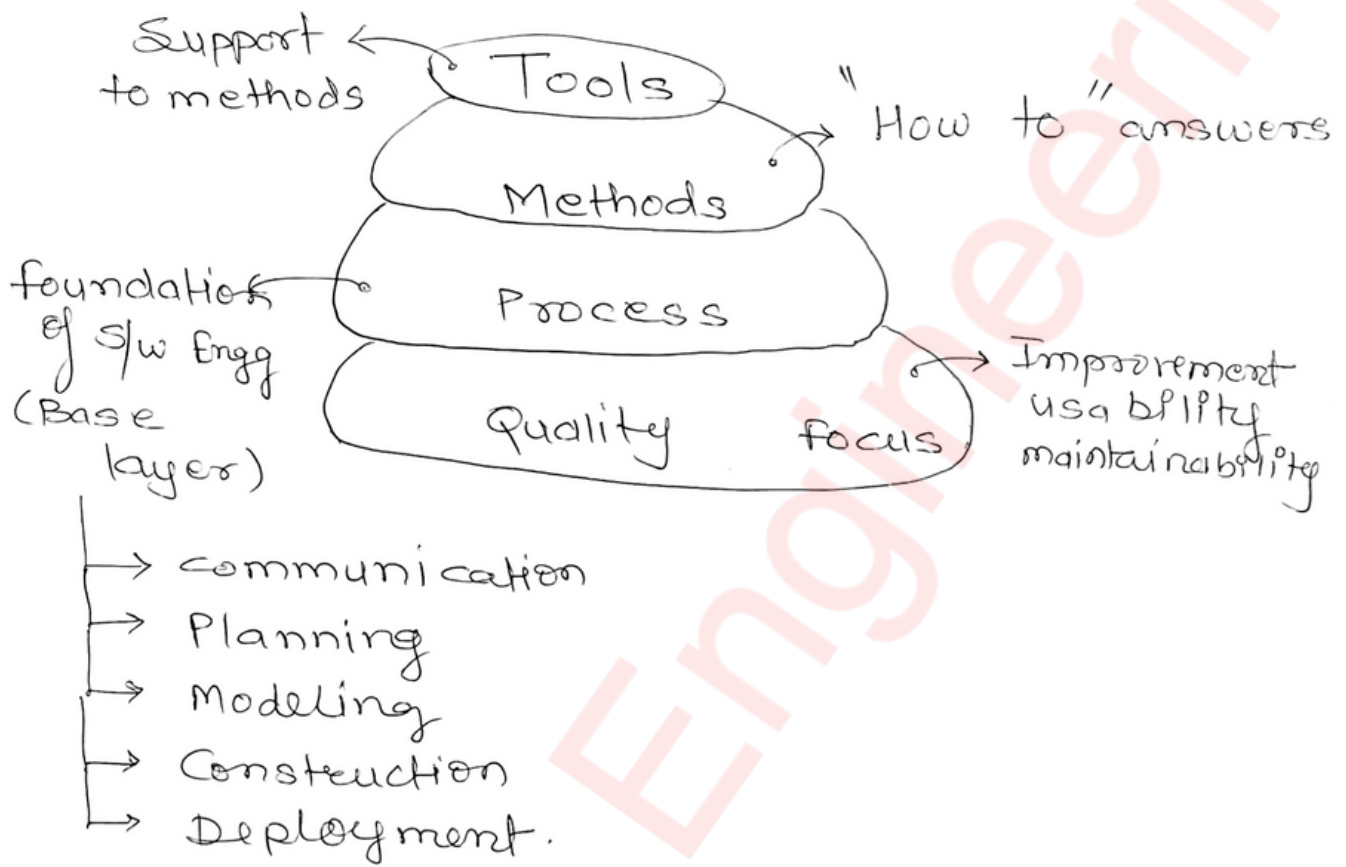
II Customer Myths

- ① General statement of intent is enough to start writing programs, the details can be filled later
- ② s/w requirements continually change but change can be easily accommodated because s/w is flexible.

III Practitioner Myths

- ① Once the program is written, the job is done.
- ② There is no way to achieve or assess system quality, until it is "running".
- ③ The only deliverable product is the working program
- ④ s/w engg creates voluminous & unnecessary documentation & slow down the s/w development.

◦ S/w Engg :- A Layered Technology.



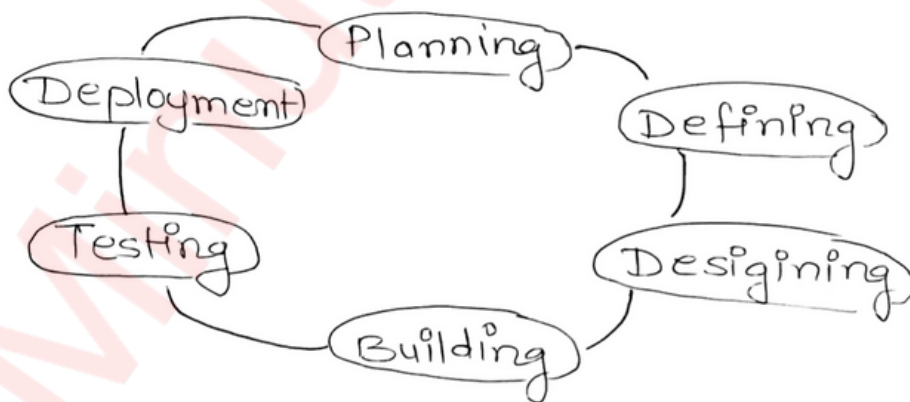
◦ Problems / challenges in s/w development.

- ① Requirements: (?)
 - Incomplete
 - Ambiguous
- ② Time & Budget: (X)
 - Unrealistic
 - Jaldi ka kaam shaitan ka hota hai
- ③ Technical skills (↓)
- ④ Change / updates (x)

• Software Quality Attributes

- Correctness [As per User Requirement]
- Usability [ease to use]
- Reliability [failure/error proof]
- Efficiency [Optimal / Best use of Resources]
- Maintainability [Ease of modification]
- Scalability [↑↓ workload / Request / User demand]
- Portability [Operating on different environment]
- Security [protection against malicious behaviour]
- Reusability [Part of one thing can be used at other]
- Modularity [Dividing a thing into multiple module & operating on them]

• Software Development Life Cycle [SDLC]



◦ Waterfall Model:

- fundamental
- Simple
- Sequential
- Small projects

No,
Going
Back

◦ Feasibility Study

◦ Requirement Analysis

◦ Design

◦ Coding

◦ Testing

◦ Maintenance

◦ Communication

◦ Planning

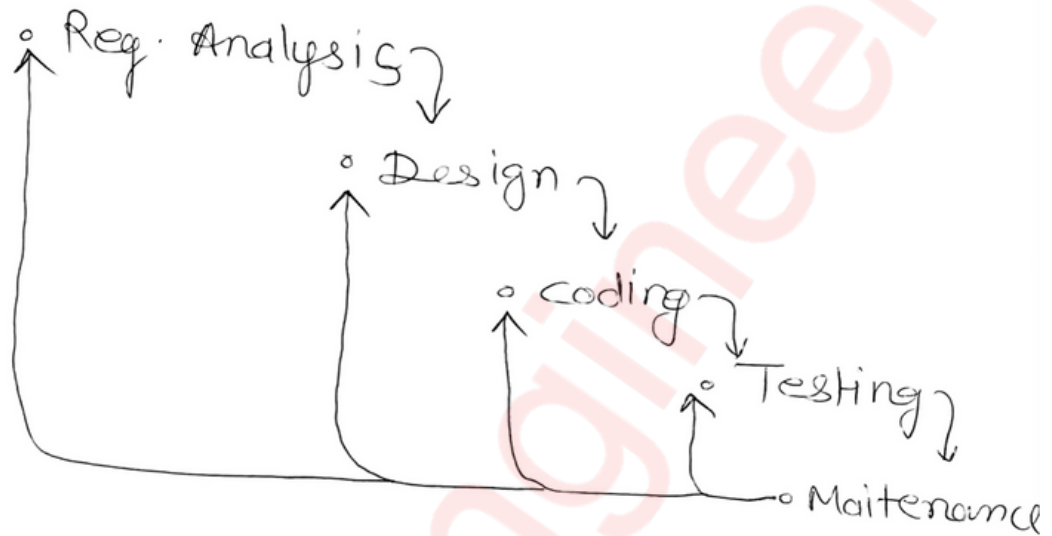
◦ Modeling

◦ Construction

◦ Deployment

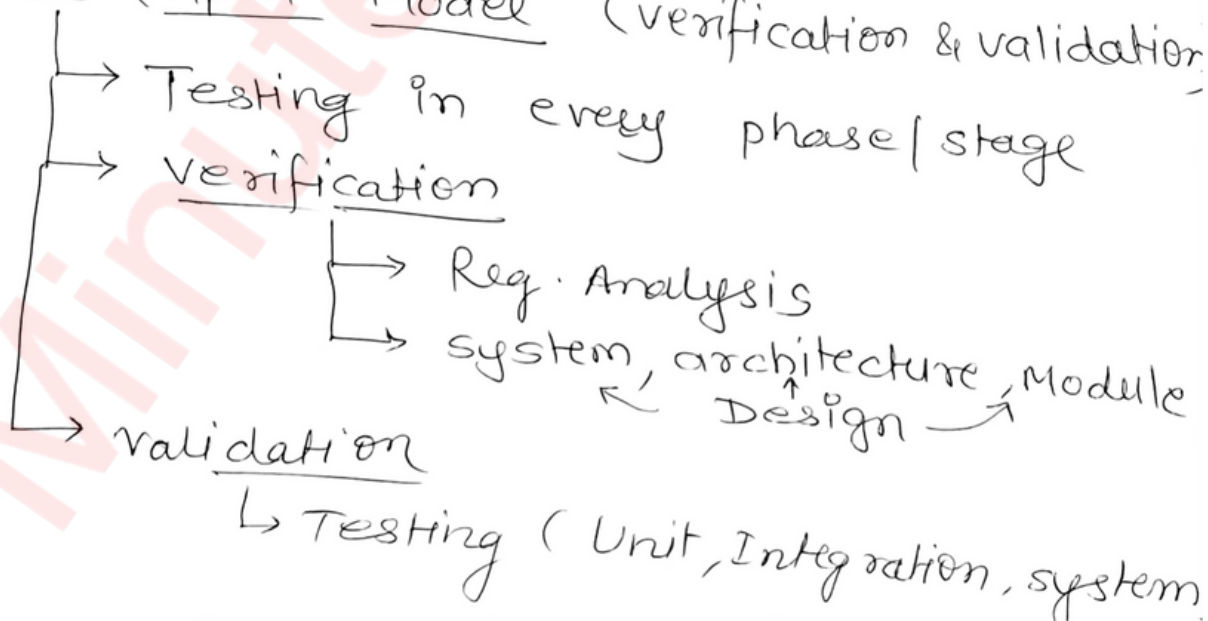
• Iterative waterfall Model

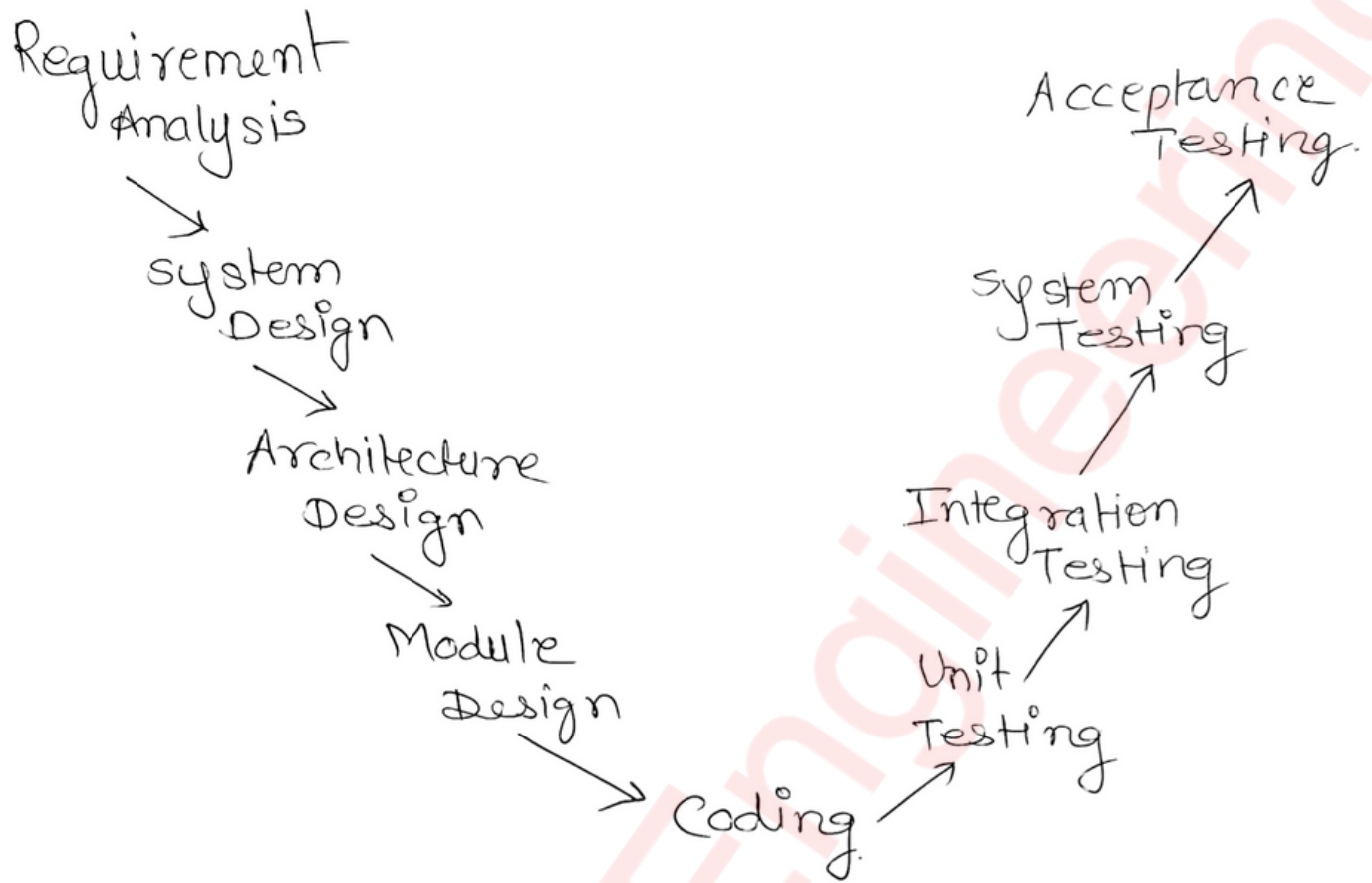
• feasibility study



- ⇒ Here You can "go back".
- ⇒ Small projects
- ⇒ serial (✓)
- ⇒ fixed Requirement Nature

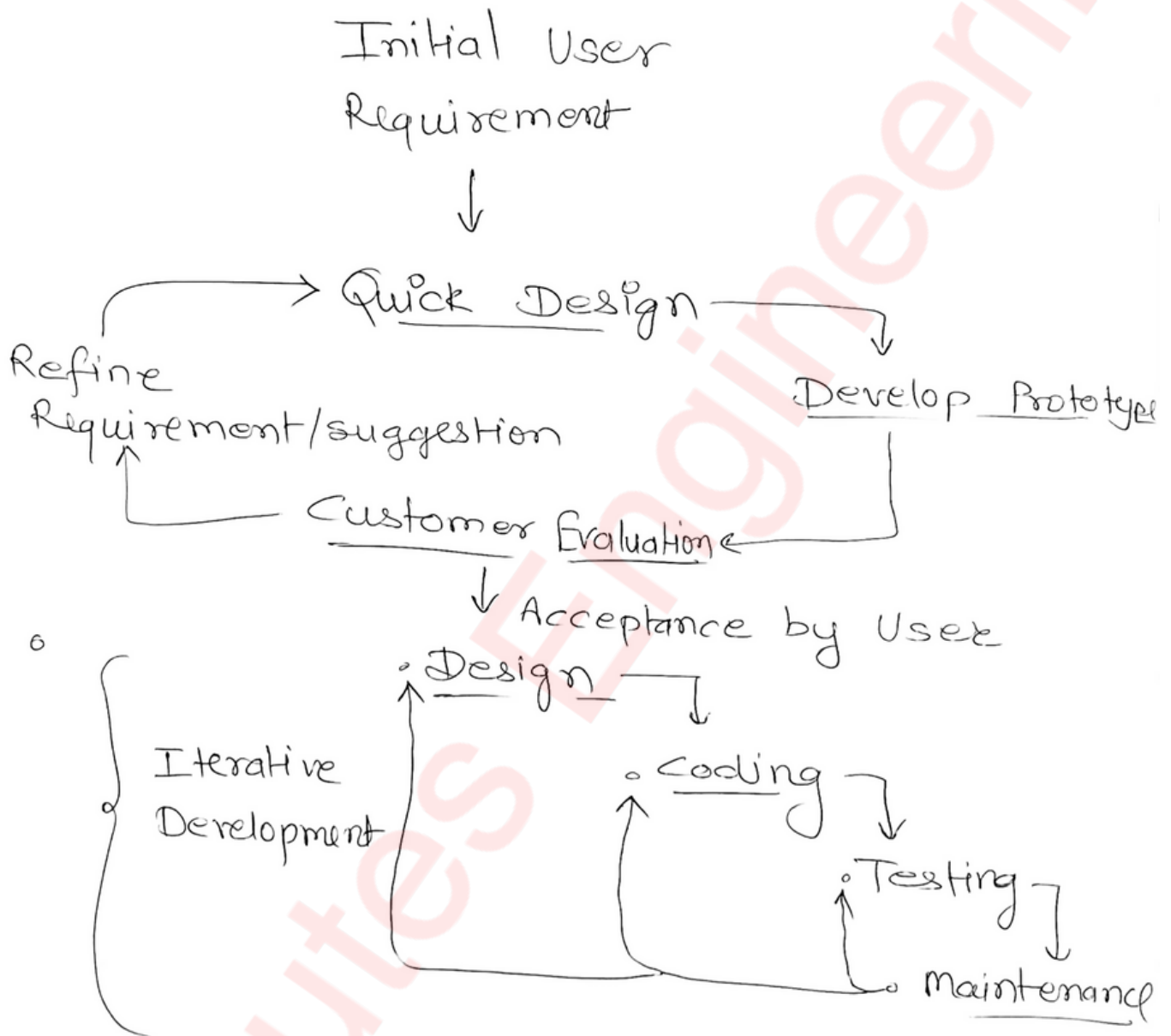
• V shaped Model (verification & validation)





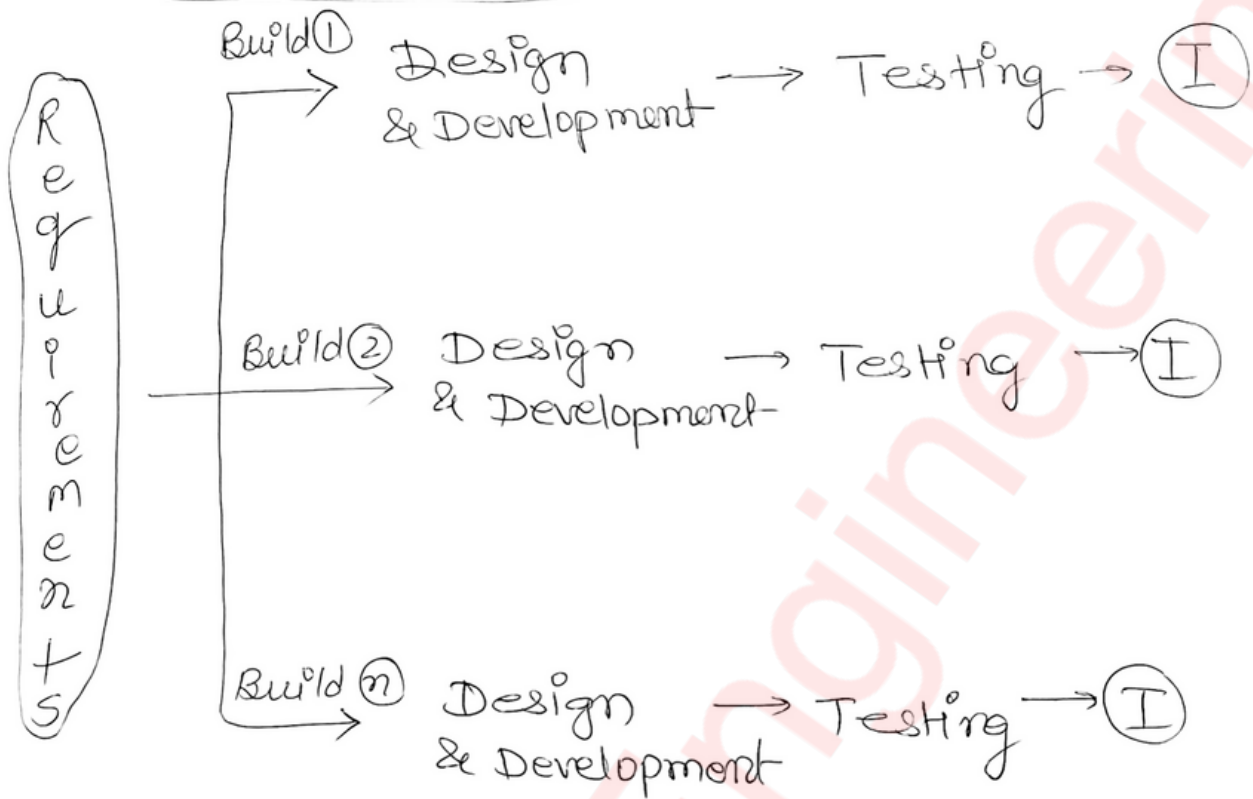
- Use when requirements are clear & fixed.
- Small to medium projects
- Simple & easy
- Time saving.
- 'Testing' is emphasis

• Prototyping Model

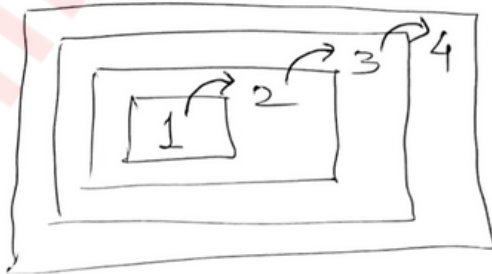


- Requirements / Idea not clear
- Cost of development ↑↑
- Can cause delays.

◦ Incremental Model

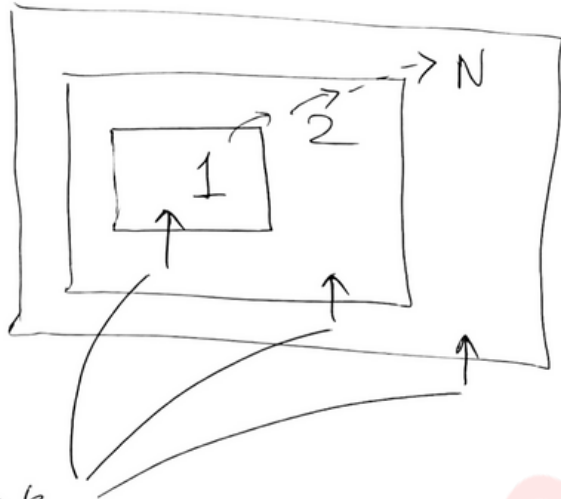


- Req. are divided into multiple modules
- Each module goes through design, development, Testing & Implementation.
- module by module [function adding]
- Can be used for medium - Large projects
- Total cost is ↑↑



◦ Evolutionary model

↳ Iterative + Incremental



feedback

- for large Projects
- Risk analysis is good
- Cost is ↑↑
- feedback is available

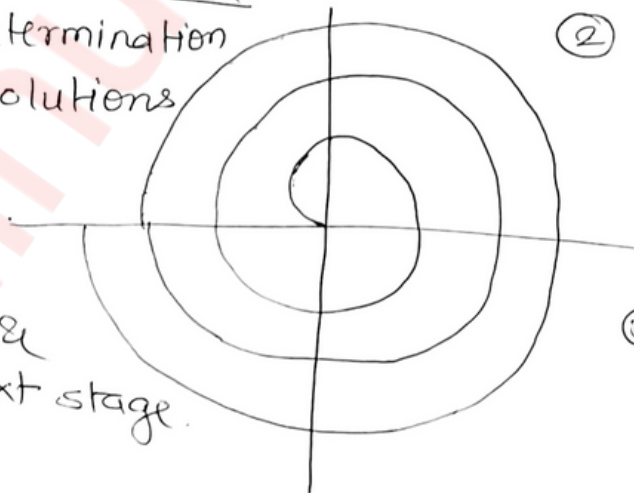
◦ Spiral model

① Obj. determination
& Alt. solutions

② Identify & Resolve
Risk

④ Review &
Plan next stage.

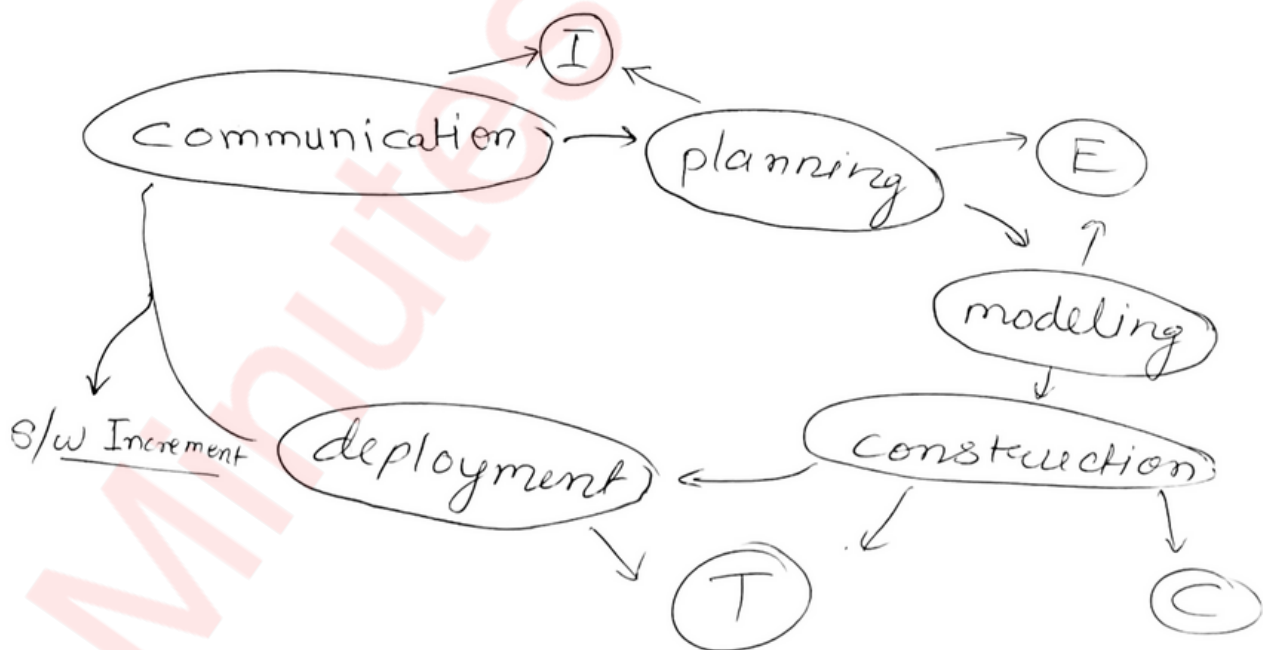
③ Develop next
version of Product



- Risk Mgmt / Handling / Analyze
- Cost $\uparrow\uparrow \propto$ Radius of spiral model.
- Can be used in Large Projects.
- Not easy & simple, its complex.
- Customer satisfaction.

◦ The unified Process

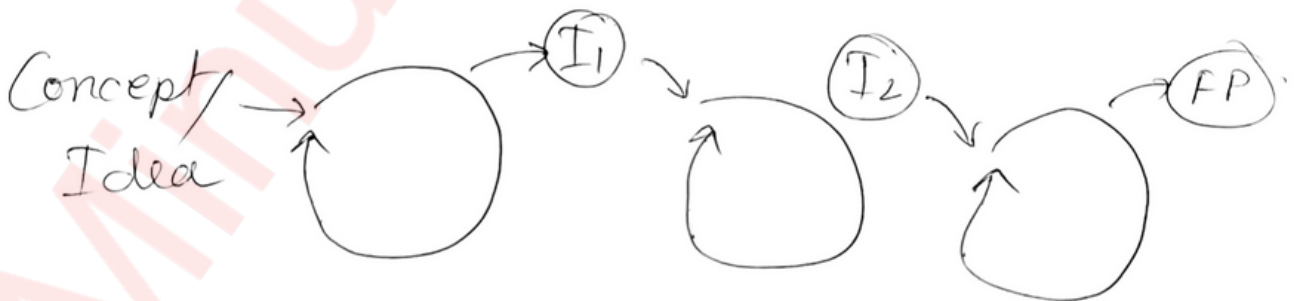
- Inception
- Elaboration
- Construction
- Transition.



◦ "Agile" S/w development.

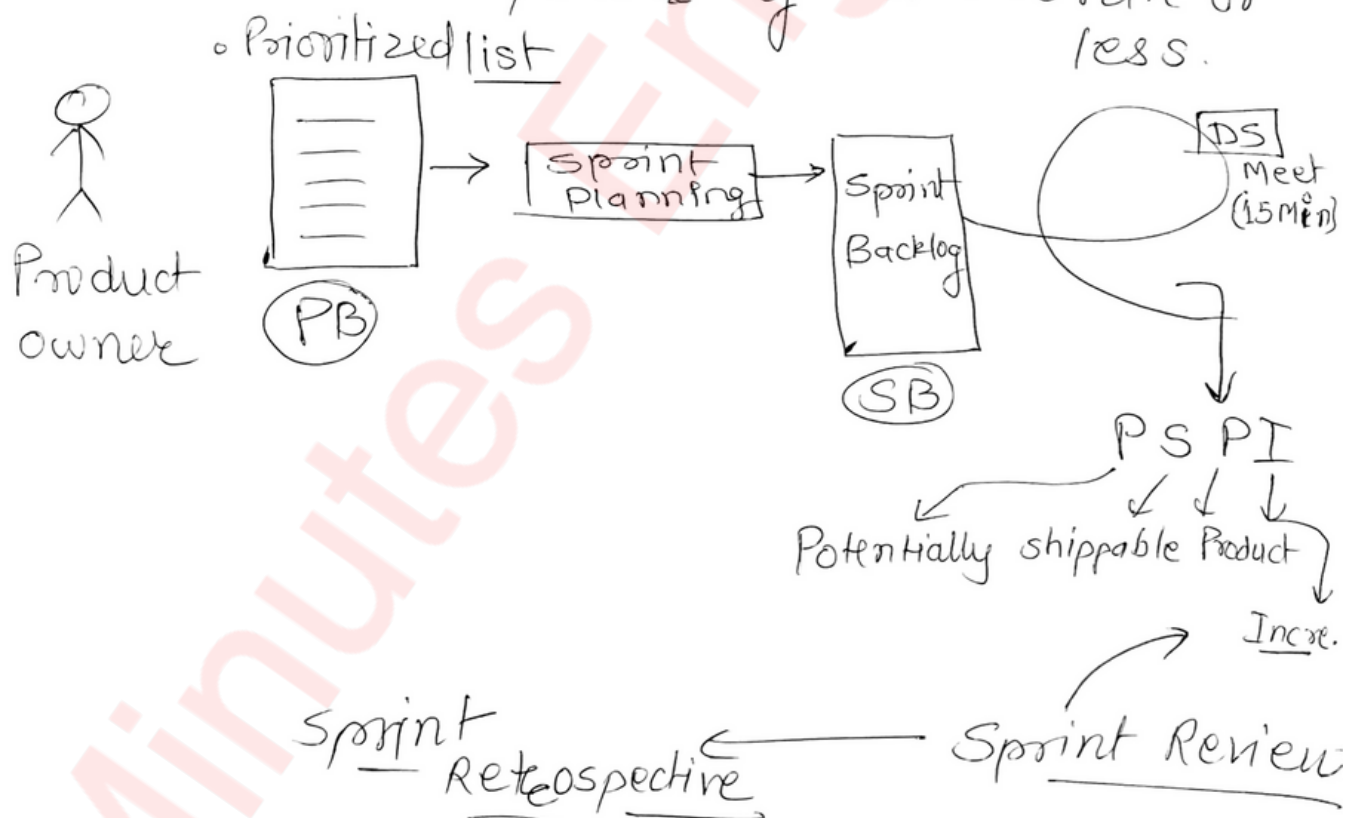
- Quick & accurate
- flexibility & collaboration
- Principles

- Communication & Collaboration
[Don't just run behind formality & Procedure]
- focus on incremental product solution first then go for detailed lengthy documentation
- Prioritize clients ideas and requirements.
- Accept changes & be flexible.



• SCRUM Model

- Agile methodology
- Light weight, simple to understand
- Team work & Collaboration.
- fast, Quick & money efficient
- Sprint: It a duration or Time period of a month or less.



° Requirements analysis & specification

- Understand $\xrightarrow{\text{exact}}$ Req. of client
 $\searrow$ what to build
- Ambiguities & inconsistencies (x)
 incompleteness

" Communication is the key "

- Requirement changes also occur
- Time & Budget constraints
- Understanding gap.
 [Different client, different field,
 different expectation]

Questions:

- ① what's the problem?
- ② why it's important to solve it?
- ③ what are the possible solutions?
- ④ what's the I/p & o/p exactly should be?
- ⑤ what issues/difficulties may arise?
- ⑥ Budget & deadline of the project?

• Steps of Analysis & Specifications.

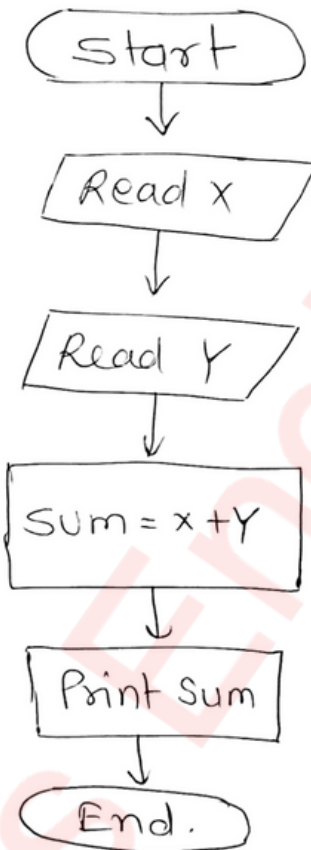
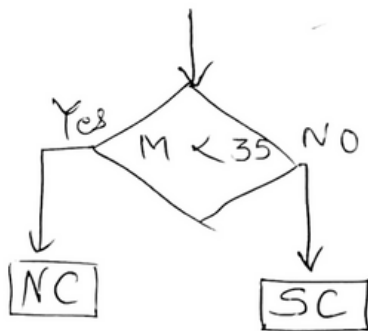
- Req. gathering
- Req. Analysis
- Req. Documentation
- Req. Review

① Req. Gathering

- Interview / meet
- Group Discussion [from Both sides]
- Delphi technique
[written QD, iterative nature]
- facilitated workshop.
[team, collaborative & creative]
- Quality functional deployment
[VIMP, IMP, NIMP]
- Role Play [Different user types]
- Use case Approach
[visual / Graphical Representation]

② Requirement Analysis

① flow chart

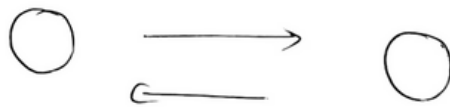
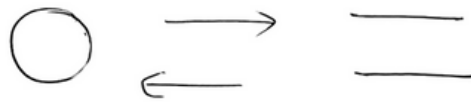
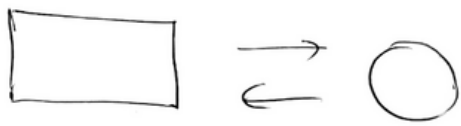


Control flow during execution of program

② Data flow diagram

- Process
- External Entity
- Data store
- Data flow





(X)

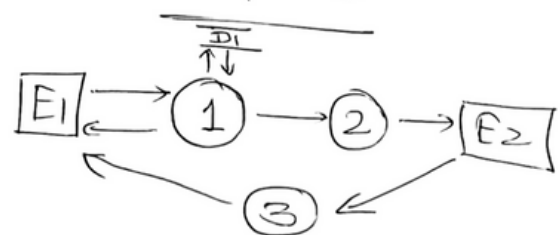


(✓)

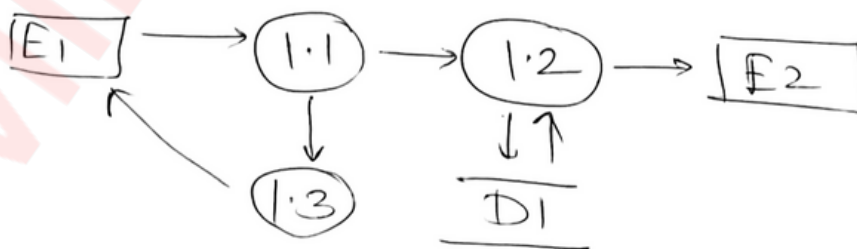
Level-0



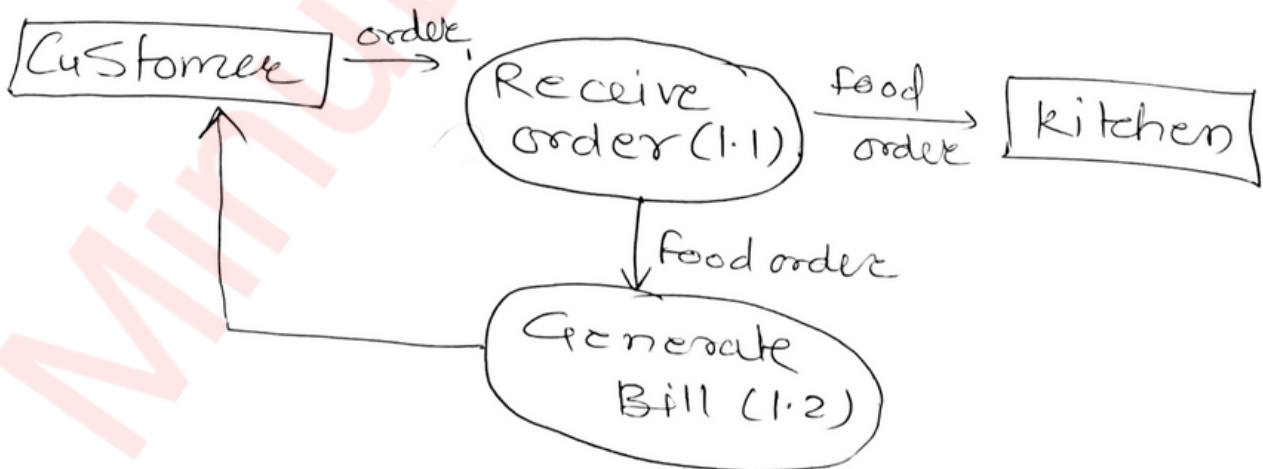
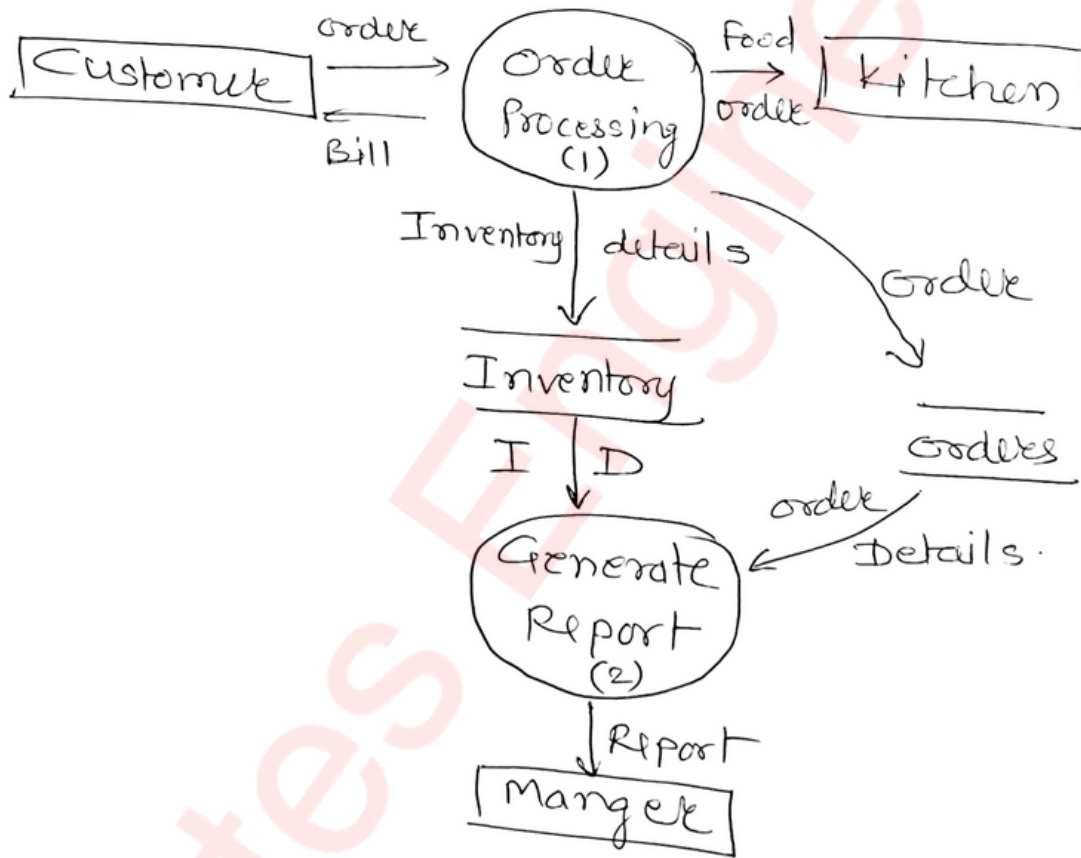
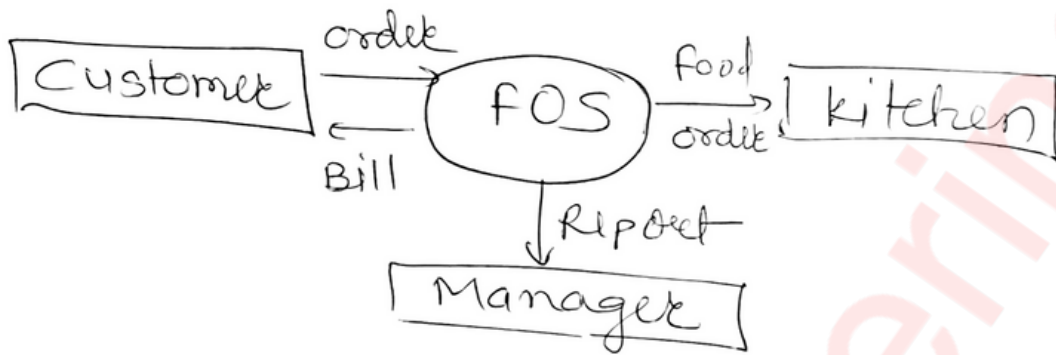
Level-1



Level-2



Eg:



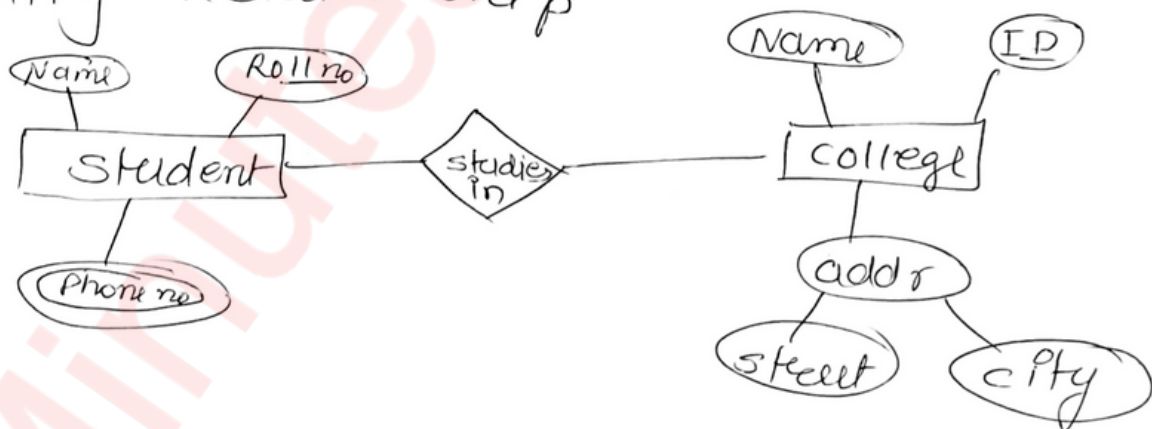
III Data Dictionary [lists all data items in DFD]

- Name of item
- Aliases
- Description
- Range of values. [1-10, 1-50, 1-100]

- It clears out the ambiguity
- Provides standard terminology for data.

IV ER Diagram

Entity Relationship



⑤ Decision Table

⇒ Cause-effect table

I/p condition o/p action

"If - then - else"

eg:

<u>Conditions</u>	<u>R1</u>	<u>R2</u>	<u>R3</u>
C1			
C2			
C3			
<u>Actions</u>			
A1			
A2			

③ Requirement Documentation :

- Req. Gathered ✓
- Req. Analyzed ✓
- Now we need to document them in some standard format (IEEE 830)
- Here our SRS will be as per the globally accepted standard.

• Benefits of IEEE 830

- ↳ Promotes → clarity
 - ↳ consistency.
 - ↳ completeness

- I) Introduction
- II) Overall Description
- III) Specific Requirements
- IV) Update Mgmt
- V) Document Approval.

④ Requirement Review

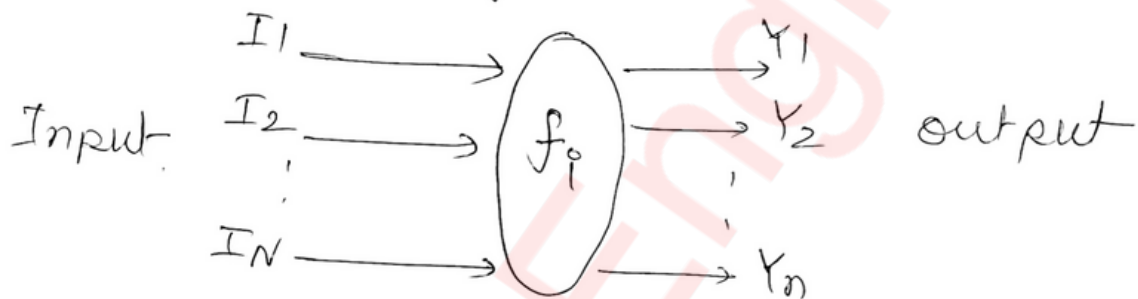
→ "final call / step / phase"

eg: final check after completely writing the answer sheet.

◦ Imp parts of SRS Documents

- functional Req. of system
- Non functional Req. of system
- Goals of Implementation

① functional Req.



"specifies the functionalities req. in system"

② Non-functional Req.

- Deals with characteristics of system that cannot be expressed as fn.
eg:- Usability, maintainability of system.

③ GDI (Guidelines/suggestions regarding development)

- Develop the sw in such a way that future Goals are also meet.

• Properties of Good SRS

- ① Complete
- ② consistent
- ③ Unambiguous
- ④ verifiable
- ⑤ modifiable
- ⑥ Testable
- ⑦ Relevant
- ⑧ Understandable & Readable

• Without SRS

- wrong system developed
- Doubts at every step
- Difficulty in flow understanding.
- Time & Budget waste.

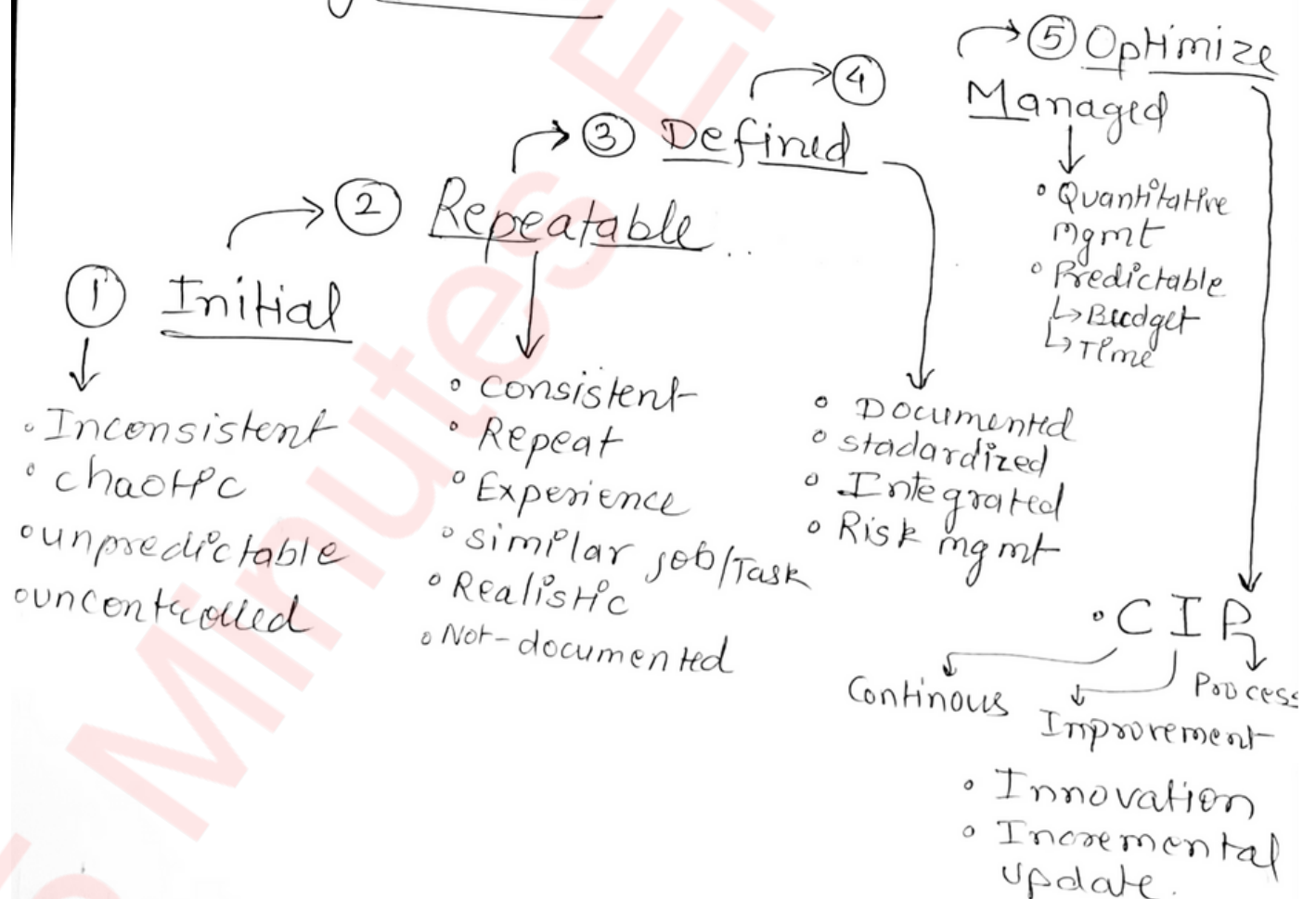
◦ Capability maturity model (CMM)

↳ Improve / Enhance software process
To get best/optimal Quality s/w

"Dhoni's mindset"

↳ focus on process
as depends on process

◦ Maturity Levels



◦ Software design $\begin{cases} \rightarrow \text{I/p} : \text{SRS} \\ \rightarrow \text{o/p} : \text{s/w design doc} \end{cases}$

- Interface Design
- Architectural Design
- Detailed Design

◦ Good s/w design doc characteristics

- ① Must have all client Req. as per SRS
- ② Must be free from conflicts, ambiguity and incorrectness.
- ③ Must explain each & every feature of s/w clearly.

① Interface Design: Interaction betⁿ System & environment

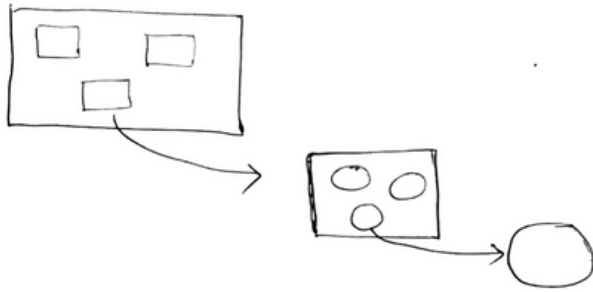
- System Internal is ignored
- Just focus on I/p to system & final o/p.

② Architectural Design: Components of system

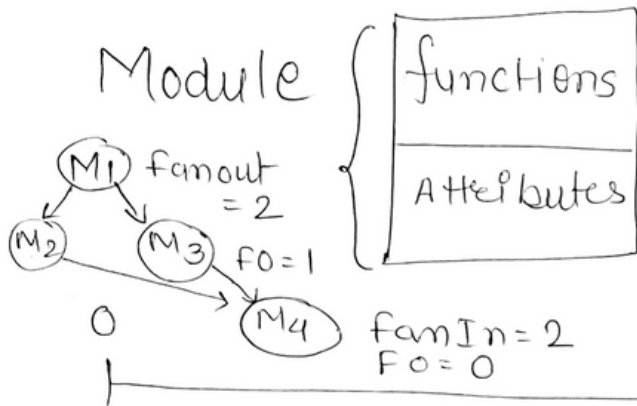
- their interfaces, responsibilities
- Interaction betⁿ them
- Internal of components are ignored.

③ Detailed Design

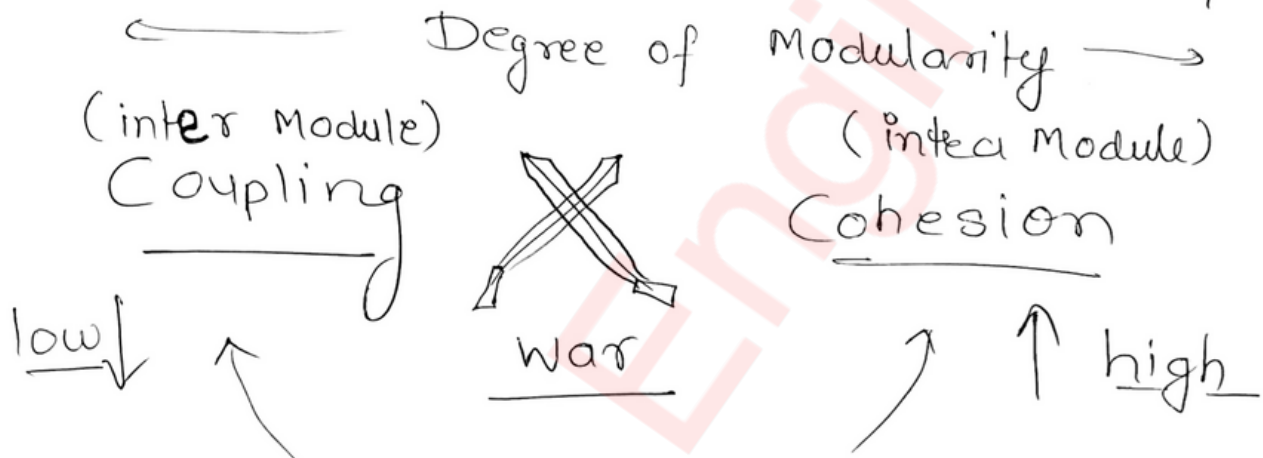
- ~> focus is on the internal elements of all components of the system
- ~> Internal Algorithms & data structure



Modularity:

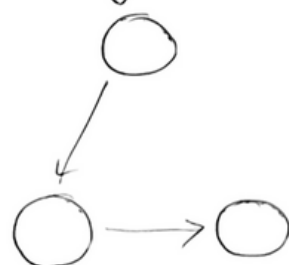
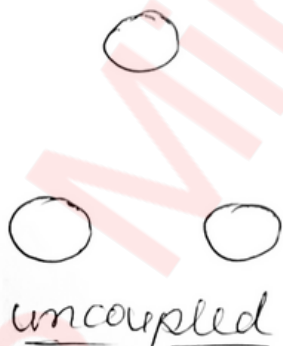


- Independent
(Complexity is Reduced)
(Its easy to manage)
(Not much)

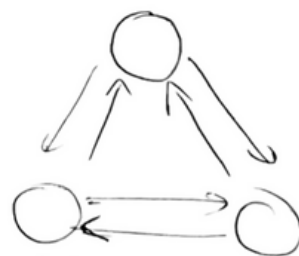


Parameters using which we try to understand the quality of modularity in design

① Coupling : The measure of Interdependence of one module on other.



Loosely coupled



Highly coupled.

Types of Coupling.

- Data (Optimal / Best)
- Stamp
- Control
- External
- Common
- Content (worst)

① Data : Data passing communication
in betn 2 modules.
(Call value)

② Stamp : Communicate / interact by
passing data structure
(call by reference)

③ Control : Control info or logic or statements
One module is trying to control
functionality of other.

④ External : Coupling in modules on external
matter like H/W, CPU, Buses.

⑤ Common : Shared / common data.
[critical section, race condition (os)]

- Content: Subset / part / portion.
when one module is a part of other module.
→ Data can be modified.
Its worst.

* Cohesion: The measure of the functions strength.
(Intra module).



Types of Cohesion

- function (High / Best)
- Sequence
- Communicational
- Procedural
- Temporal
- logical
- Coincidental (low / worst)

① functional: different f^n s come together collaborate & work to perform a single task/ f^n in a Module.

② Sequential: $f_1 \rightarrow f_2 \rightarrow f_3$
• One's O/p is I/p for other
• flow & control dependency.

③ Communicational: 2 f^n operate on same I/p data / data structure

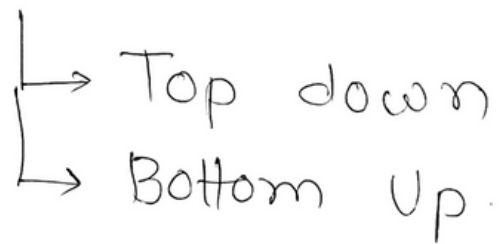
④ Procedural: flow control, it is part of a procedure to be followed.

⑤ Temporal: (Time) A module having temporal C, all the f^n must be executed in same time span.

⑥ Logical: Similar operation performed.
(logically related)
(functionally different)

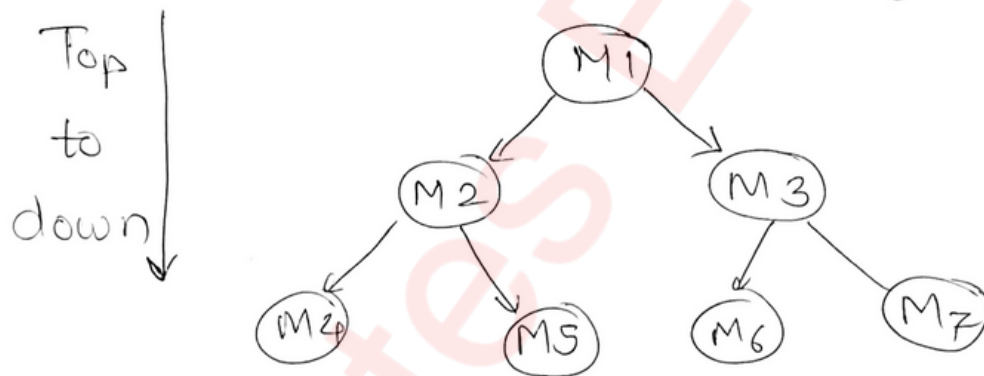
⑦ Coincidental: Random or some kind of coincidence (worst).

• Design Approach



• Top down design approach

- Divide & Conquer.
- Dividing a big problem into small problem further more repeatedly till we solve it easily.
- You need to have entire problem requirements ^{Problem statement}
- Small or medium size project.



• Bottom Up design approach

- Initially no need to have understanding of entire s/w req. / product req.
- Go from low level fⁿ/modules to higher ones.
- Eg: Engineering exam last night study.

◦ Estimation

→ Project Planning Process

- Before development
- focus on activities req. for successful completion of project.

◦ Estimating

- Size of Project
- Cost
- Time / Duration
- Effort.

Precise &
accurate

Estimates

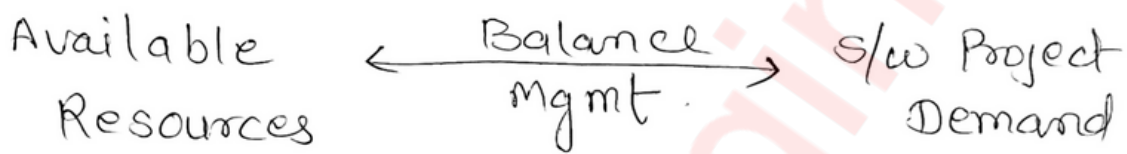
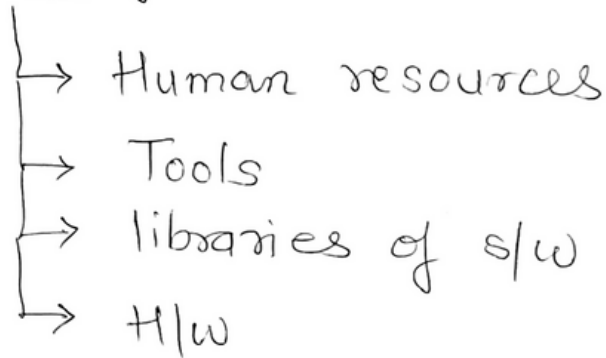
leads
to

successful
completion
or mgmt of
Project.

for cost estimation

- size of s/w
- H/w
- Tools & licenses
- skilled personnel
- Travel & communication
- Training & support.

Resource Mgmt



- ① Imp / Immediate Resources at each phase must be determined. to make them available.
- ② Resoure request on demand & de allocating them whay they are no lo get required.
- ③ Reuse the s/w Resources
 - Requirements
 - Design
 - code
 - Test cases.

Estimation Techniques

- Empirical
- Heuristic
- Analytical

① Empirical :

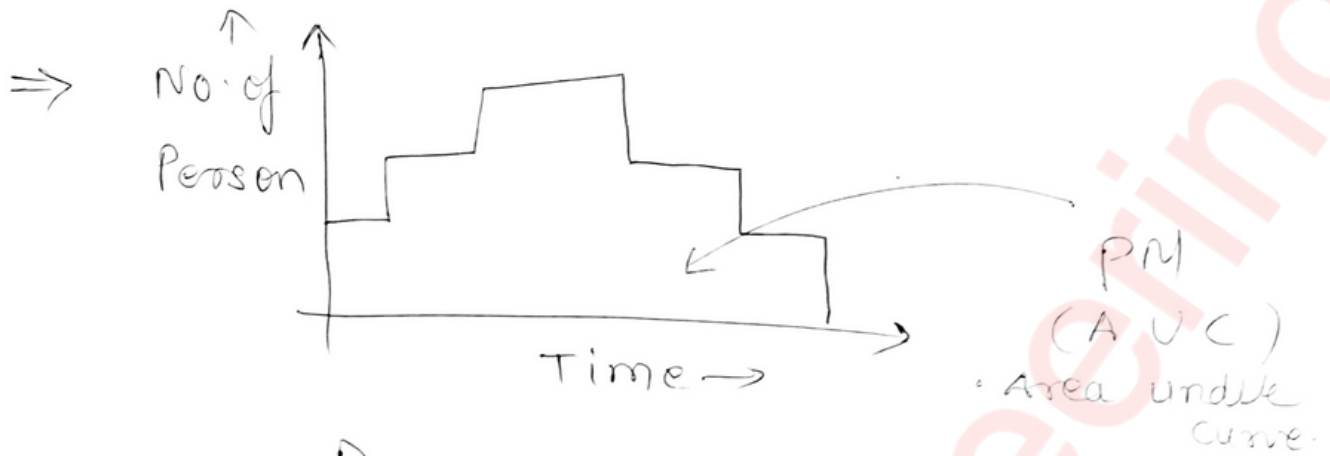
- Guesses or Assumption
- Prior experience on similar projects
- historical/previous product/project Data.
- Common sense based.
- • Expert Judgment [one or pair]
• Delphi cost estimation.
[Group or team work]

② Heuristic

→ C O C O M O
↙ ↓ ↓ ↓ ↓ ↘
Constructive cost model.

- Mathematical eqⁿs involving the project parameters
- Independent parameter first.
then dependent parameter.

→ CO CONTO



Person month curve/plot

So, effort estimate

	a_1	a_2
→ organic	$2.4 (KLOC)^{1.05}$	PM
→ semi-detached	$3.0 (KLOC)^{1.12}$	PM
→ Embedded	$3.6 (KLOC)^{1.20}$	PM

2. Time estimate

	b_1	b_2
→ organic	$2.5 (E)^{0.38}$	Months
→ semi-detached	$2.5 (E)^{0.35}$	Months
→ Embedded	$2.5 (E)^{0.32}$	Months

Q:- Project type: semi-detached.

$$\begin{aligned} \text{LOC} &= 50,000 \\ &= 50 \text{ KLOC} \end{aligned}$$

∴ ① Effort estimate

$$\begin{aligned} &= a_1 (\text{KLOC})^{a_2} \\ &= 3.0 (50)^{(1.12)} \\ &= 239 \text{ PM} \end{aligned}$$

$$\begin{aligned} \text{② } T &= b_1 (E)^{b_2} \\ &= 2.5 (239)^{0.35} \\ &= 17 \text{ months.} \end{aligned}$$

$$\begin{aligned} \text{③ Productivity} &= \frac{\text{KLOC}}{(P) \quad E} \\ &= \frac{50}{239} \\ &= \underline{0.209} \end{aligned}$$

◦ Intermediate COCOMO

- Basic COCOMO assumes that $\textcircled{E}$ & $\textcircled{T}$ are fⁿ of size alone.
- But besides size other project parameters.

eg: LOC = 30,000

Type = Embedded.

Parameter: Reliability must be high

- Product Attr
- Personal Attr
- Project Attr
- Computer Attr.

(very low, low, normal, high, very high)

So modified formula = $\alpha_1 (KLOC)^{\alpha_2} \times (EAF)$

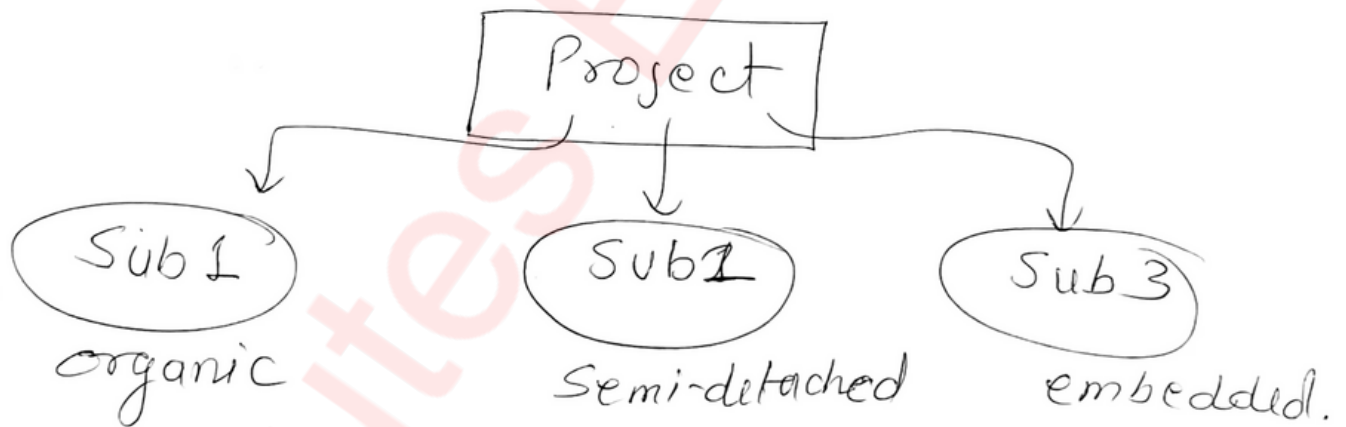
$$= 2.8 (30)^{1.20} \times 1.15$$

$$= 191 \text{ PM}$$

$$T = b_1 (E)^{b_2} = 2.5 (191)^{0.32} = 13 \text{ Month}$$

◦ Complete Cocomo

- Intermediate Cocomo consider a s/w project as a single homogeneous entity.
- But large projects/products/systems are made up of several small sub-systems.



E: Estimate of sub-systems.

$$E(\text{Sub 1}) + E(\text{Sub 2}) + E(\text{Sub 3})$$

• Software Testing

- To find Bugs/faults/Errors in Code.
- With min resources, cost & time we can Identify max bugs.

Verification

- 1st
- "Are we building the product/thing right?"
- Prevent errors
- Phase by Phase
- By Developer
- Includes Unit testing & Integration testing & Reviews
- Static nature

Vs

Validation

- 2nd
- Are we building the right product/thing.
- Detect errors
- Final Product.
- By Tester
- Includes Black Box & white Box testing.
- Dynamic nature

◦ Principles of s/w Testing

- Identify as many bugs as possible
- Early testing saves your resources
- Start with a proper test plan
- As per user requirements
- focus more on heavy Bug Areas.
- Go for functional as well as non-functional Testing.

◦ Unit Testing : Each Source code Module is Tested.

- Hence also called as module testing.
- Early phase testing.

◦ Integration Testing

- Incremental (Top-down & Bottom Up)
- Non-Incremental.
(Big Bang)

- Interaction, Connectivity, & Data flow betⁿ unit tested modules

◦ System Testing

- Whole / Complete System
- Don't focus on internal work
- focus on I/p & o/p
- Black Box Testing
- Verifies the overall functionality of system.

◦ Acceptance Testing (UAT) α, β

- final Testing phase
Before s/w deployment
- focus on ease of use
for Best user experience
- performed by clients
or end users.

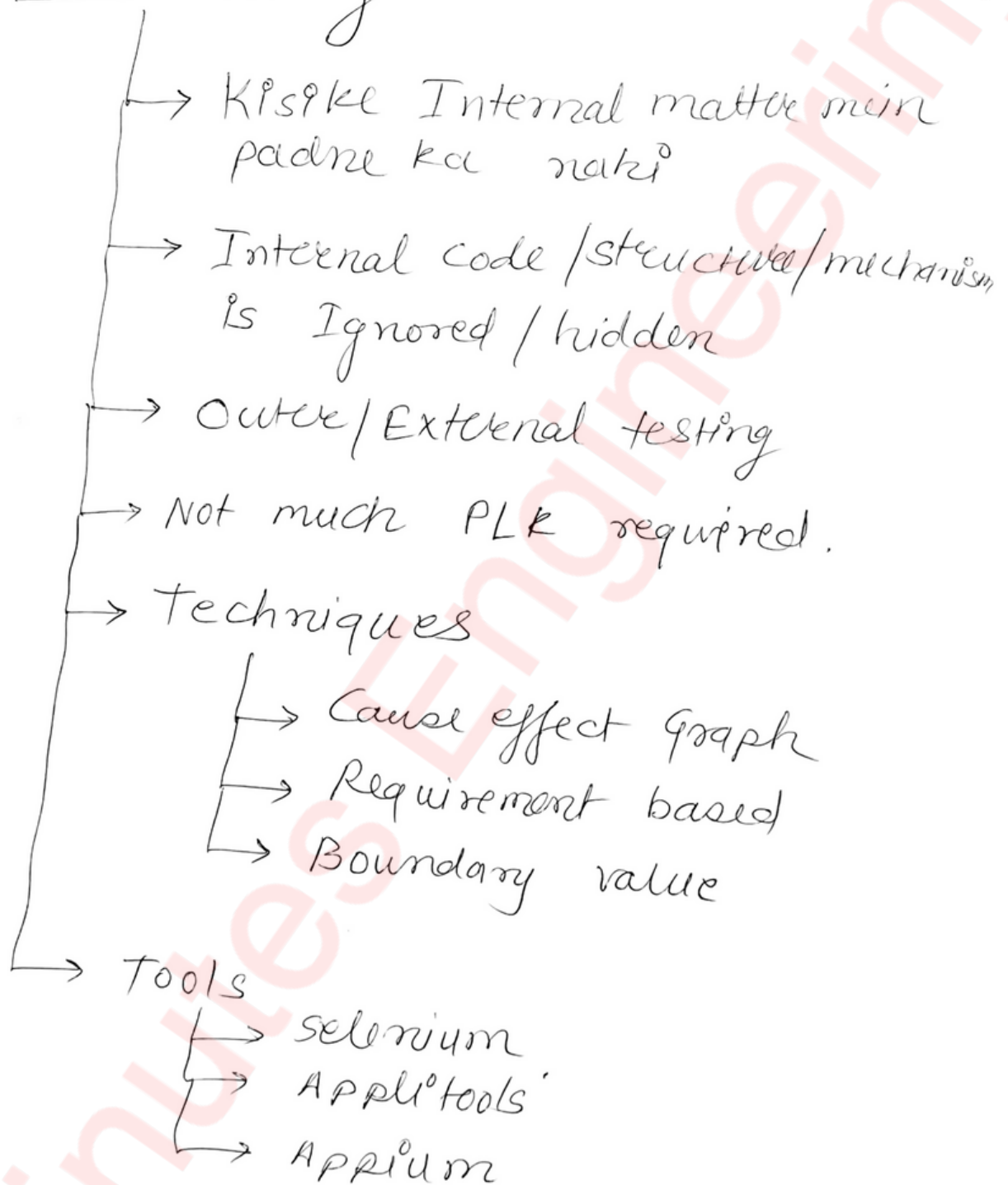
- Performance Testing
- Load Testing
- Stress Testing
- Scalability Testing

Non functional

• White Box Testing

- Glass, transparent & structural & clear Testing.
- focus on internal structure Testing.
- used/applied on unit & integration level testing.
- Techniques
 - statement coverage
 - Branch coverage
 - Dataflow testing
 - path coverage.
- Tools: Sqlmap, Nunit, VeraUnit

° Black Box Testing



• Software maintenance

- Starts once the sw product is deployed & users are using it.
- Update / Upgrade / Improve.
- Bug fixing
- Optimization
- Light weight
- delay / lag elimination
- It covers majority time / efforts in SDLC.
- Types
 - fixing / corrective
 - Perfective
 - Adaptive
 - Preventive

• Software Re-Engineering

→ Examine & Update the existing sw to enhance its performance.

→ Something more than maintenance.

→ Reverse / Backward Engineering.

Product code → module / Component → Design
↓
Requirement.

→ Restructuring.

↳ Preserve the core function but reorganize/refactor the existing code to make it more optimal to use & understand.

→ Forward Engineering

↳ follow the normal SDLC for new/improved Requirements.

• Critical Path method:

- ↳ Critical task [Can't be delayed]
- ↳ Critical Path
 - ↳ Min. time required to complete the project that path covering the critical task.

→ finding critical Path.

- Identify
- Sequencing.
- Estimation
- Visualization / Diagram
- Selection.

Eg:

Task

Predecessor

Time / Duration

A

—

2

B

A

3

C

B

5

$$\xrightarrow{F}$$

EST	D	EF
TL		
LS	F	LF

← R/B

(A)

→ (B)

→ (C)

Float = LF - EF
LS = LF - D

0	2	2
A		
0	0	2

2	3	5
B		
2	0	5

5	5	10
C		
5	0	10

◦ Regression Testing

↳ Bug fixing or Updation or new feature add ons

◦ Do not change the core system function/nature.

→ Full RT : Test the entire system after update.

→ Partial RT : Test only that module where update is done.

• Risk Management

→ Problem → uncertain & Impact/loss
° event
° Condition

→ Identification } Risk Assessment
→ Analysis }
→ Planning }
→ monitoring. } Risk control.
→ Resolution }

• Reactive & Proactive RM

↓
React to Risk/Problem

"Kya gadbad hogayi?"

→ Problem occurred
→ Risk that has
↑↑↑ high chances
to occur

↓
Prevent the Risk

"Kya gadbad ho sakti hai?"

→ future Risk
→ Prevention is
better than
cure.

Types of Risk

- Budget Risk
- Schedule / Time Risk
- Technical Risk
- Operational Risk
- Business Risk.

• Risk Assessment

↳ Identify, analyse & plan or prioritize the risks.

↑ more damage ↑ High priority
(severity)

↓ less damage ↓ less priority.

Risk { frequently occurring
(probability of occurrence)
Rare occurrence.

◦ Impact Assessment Matrix.

Probability (P)

		Severity (S)			
		④	③	②	①
5		<u>20</u>	<u>15</u>	<u>10</u>	<u>5</u>
4		<u>16</u>	<u>12</u>	<u>8</u>	<u>4</u>
3		<u>12</u>	<u>9</u>	<u>6</u>	<u>3</u>
2		<u>8</u>	<u>6</u>	<u>4</u>	<u>2</u>
1		<u>4</u>	<u>3</u>	<u>2</u>	<u>1</u>

◦ Risk Control

- Try to Reduce the risk occurrence probability by some actions.
- PLAN (A) PLAN (B) PLAN (C).
- Avoid, Transfer or Reduce.
- Monitoring is a process that is carried throughout the project. (Continuous evaluation).
- Risk mitigation: Try to Reduce the impact of Risk on system.